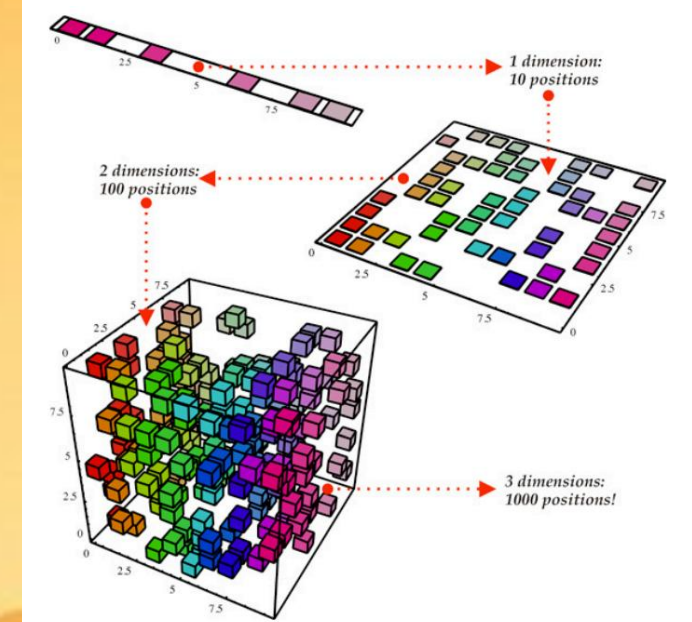


Reducción de dimensionalidad

Joan Raygoza



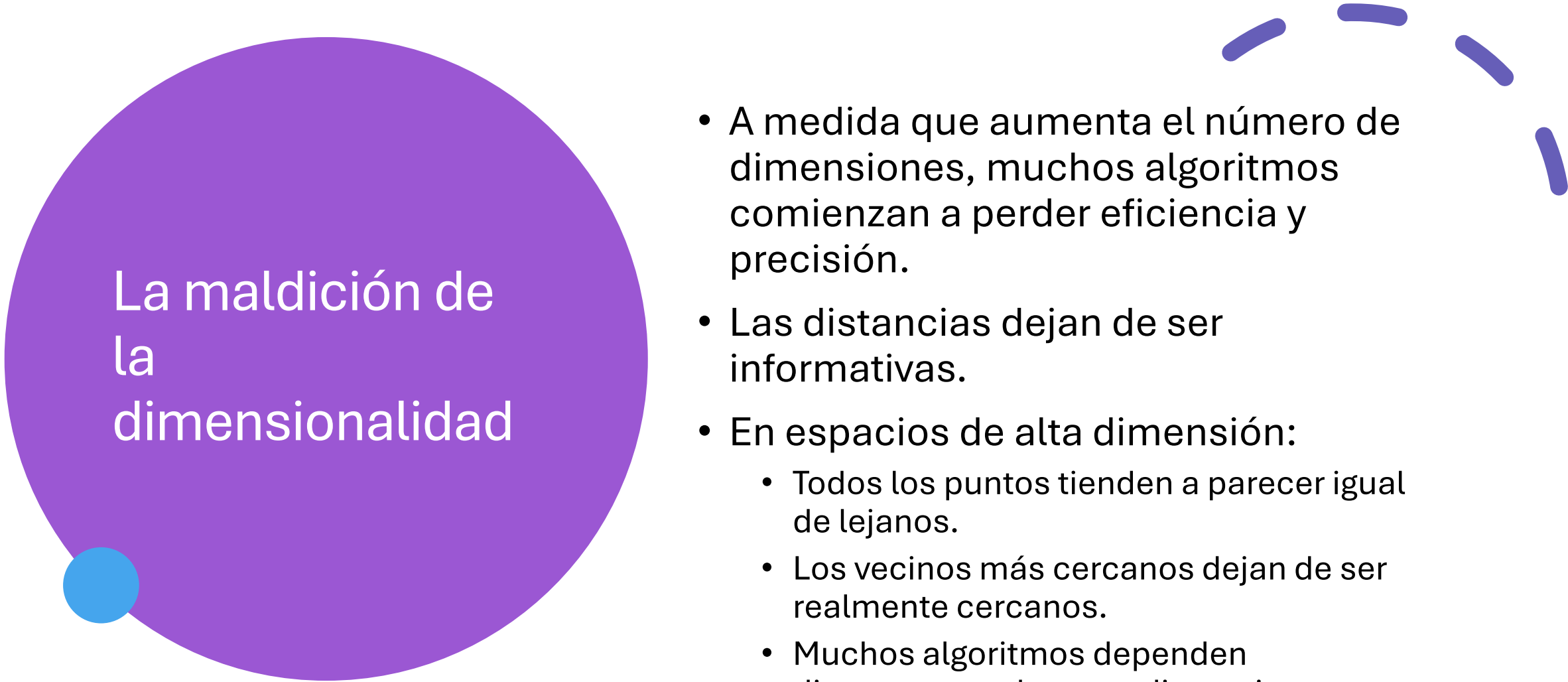
Datos y Problemas

¿Qué es la reducción de dimensionalidad?

- La **reducción de dimensionalidad** consiste en transformar un conjunto de datos con muchas variables en una representación con menos dimensiones, preservando la mayor cantidad posible de información relevante.
- Una **dimensión** corresponde a una característica o variable del conjunto de datos.
- El objetivo es obtener una representación **más compacta, más fácil de analizar y más eficiente** para los algoritmos de aprendizaje automático.

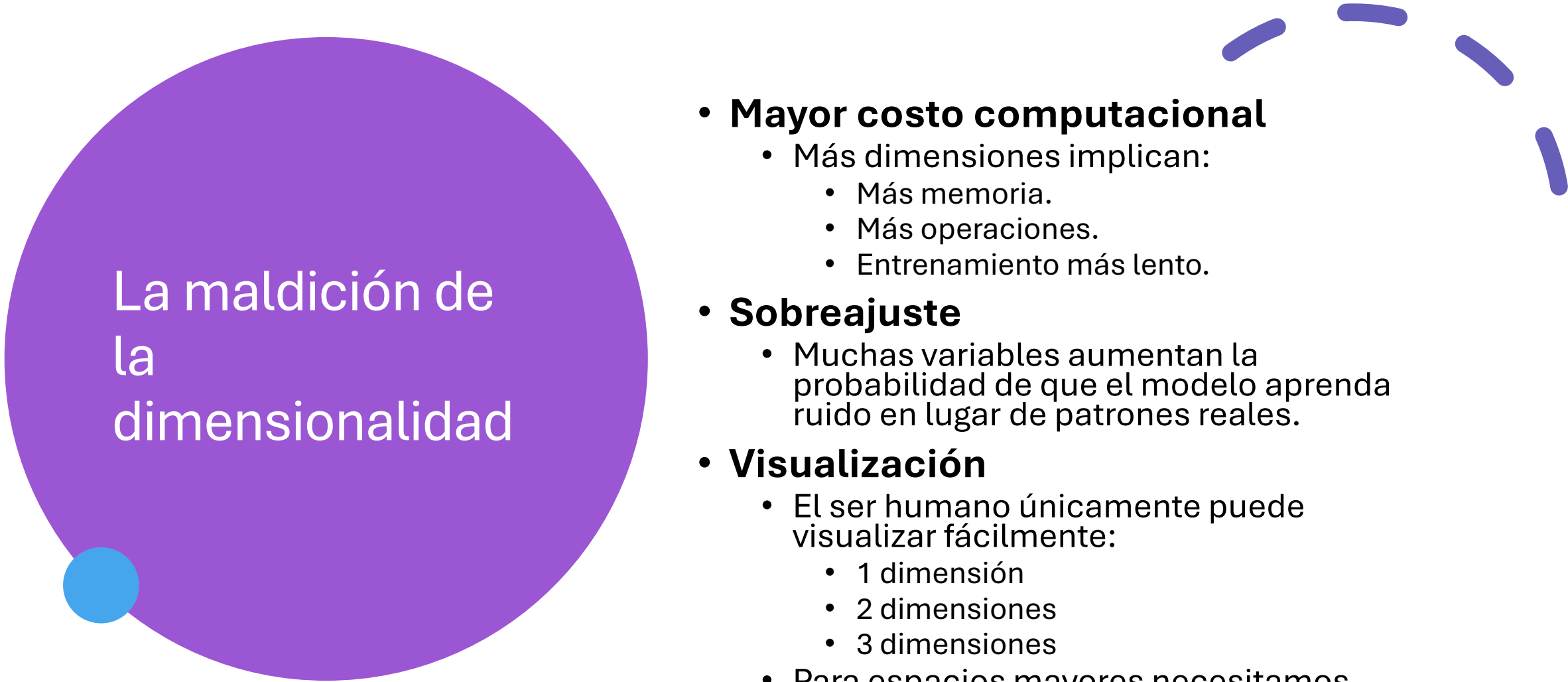
¿Qué es la reducción de dimensionalidad?

- **Ejemplos de datos de alta dimensión**
 - Texto → miles de palabras o embeddings de cientos/miles de dimensiones.
 - Imágenes → cada píxel representa una variable.
 - Audio → cientos de valores por ventana.
- **¿Por qué reducir dimensiones?**
 - Disminuir costo computacional.
 - Eliminar redundancia.
 - Reducir ruido.
 - Facilitar la visualización.
 - Mejorar el desempeño de algunos modelos.



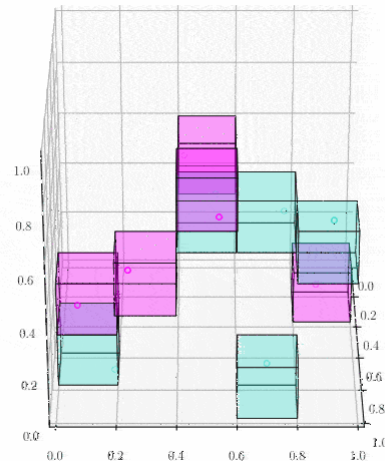
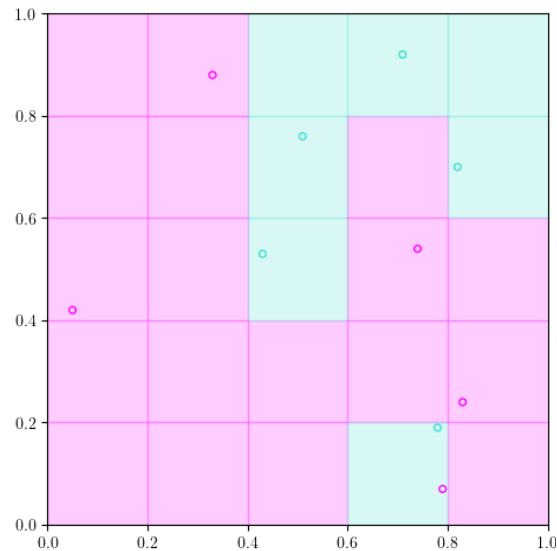
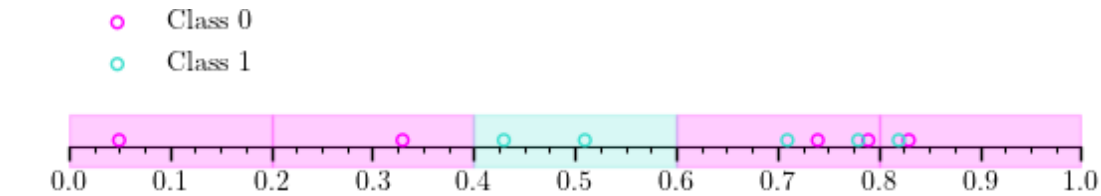
La maldición de la dimensionalidad

- A medida que aumenta el número de dimensiones, muchos algoritmos comienzan a perder eficiencia y precisión.
- Las distancias dejan de ser informativas.
- En espacios de alta dimensión:
 - Todos los puntos tienden a parecer igual de lejanos.
 - Los vecinos más cercanos dejan de ser realmente cercanos.
 - Muchos algoritmos dependen directamente de estas distancias.



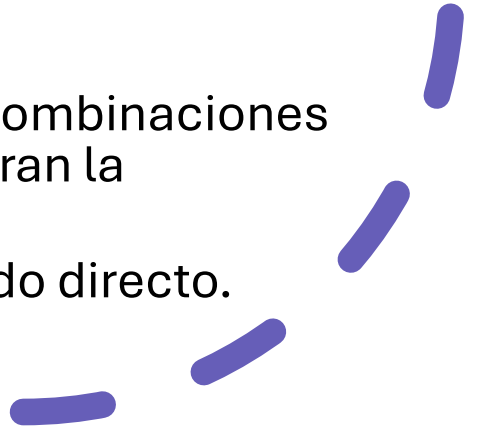
La maldición de la dimensionalidad

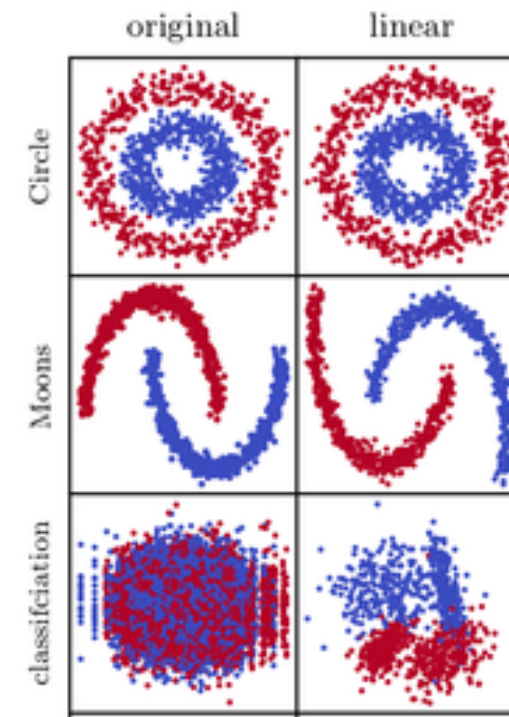
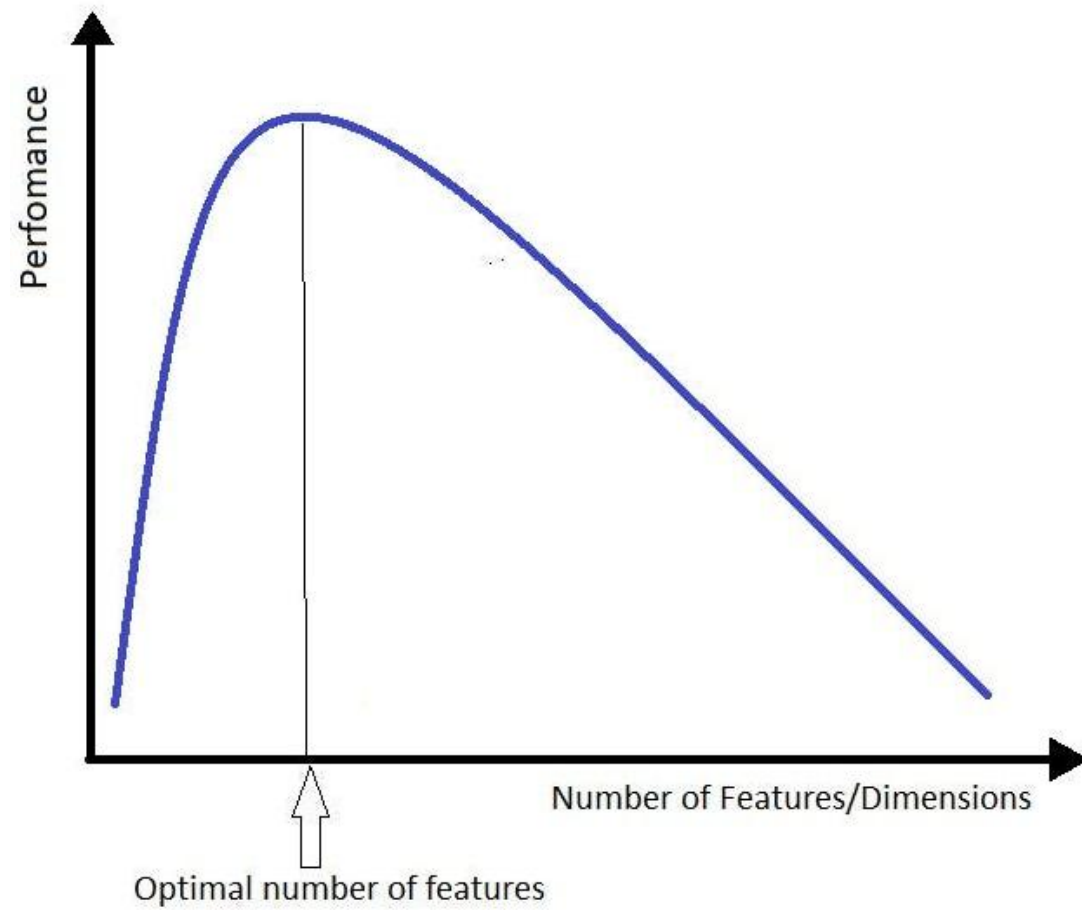
- **Mayor costo computacional**
 - Más dimensiones implican:
 - Más memoria.
 - Más operaciones.
 - Entrenamiento más lento.
- **Sobreajuste**
 - Muchas variables aumentan la probabilidad de que el modelo aprenda ruido en lugar de patrones reales.
- **Visualización**
 - El ser humano únicamente puede visualizar fácilmente:
 - 1 dimensión
 - 2 dimensiones
 - 3 dimensiones
 - Para espacios mayores necesitamos proyectar los datos.



- Los datos son generados aleatoriamente pero en alta dimensión se pueden separar fácilmente.
- A medida que aumenta el número de características, nuestros datos se vuelven más dispersos, lo que da lugar a un sobreajuste. Por lo tanto, necesitamos más datos para evitarlo.

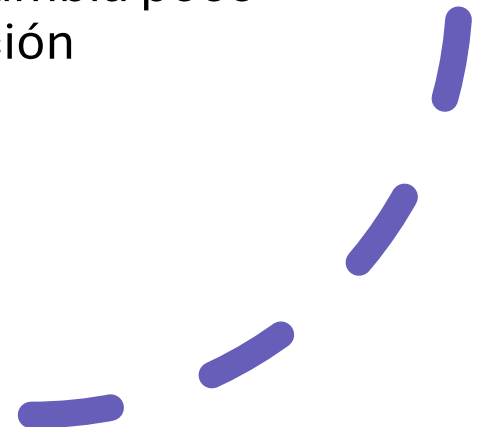
Objetivos de la reducción de dimensionalidad

- **Compresión**
 - Representar los datos utilizando menos variables, reduciendo almacenamiento y tiempo de procesamiento.
 - **Visualización**
 - Proyectar datos de alta dimensión a dos o tres dimensiones para descubrir grupos, patrones, anomalías y/o relaciones.
 - **Eliminación de ruido**
 - Las dimensiones menos relevantes suelen contener ruido o información redundante.
 - Reducir dimensiones puede mejorar la capacidad de generalización del modelo.
 - **Extracción de características**
 - Las nuevas dimensiones representan combinaciones de las variables originales que concentran la información más importante.
 - No necesariamente tienen un significado directo.
- 



Varianza e información

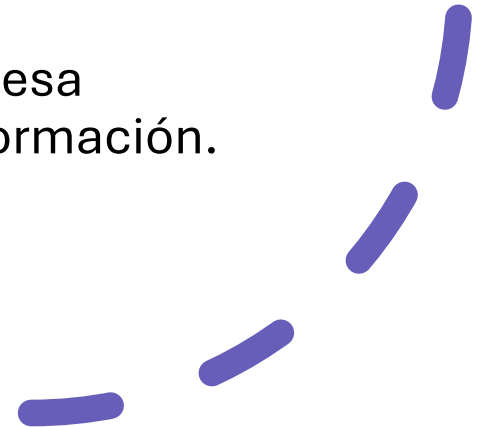
- En reducción de dimensionalidad asumimos que **las variables que presentan mayor variabilidad contienen más información útil**, mientras que aquellas con poca variación suelen aportar información redundante o ruido.
- **¿Por qué la varianza importa?**
 - Una variable con **alta varianza** distingue mejor unas observaciones de otras.
 - Una variable con **muy baja varianza** cambia poco entre muestras y aporta poca información discriminativa.



Varianza e información

- **Información redundante**

- En muchos conjuntos de datos existen variables altamente correlacionadas.
- Ejemplo:
 - Estatura (cm)
 - Estatura (m)
- Ambas contienen prácticamente la misma información.
- También ocurre con variables como:
 - Peso (kg)
 - IMC
- Reducir dimensiones permite eliminar esa redundancia sin perder demasiada información.



Métodos lineales

- Suponen que los datos pueden describirse mediante una combinación lineal de las variables originales.
- **Características:**
 - Más rápidos.
 - Más fáciles de interpretar.
 - Funcionan muy bien cuando la estructura es aproximadamente lineal.
- **Algoritmos representativos:**
 - **PCA** (Análisis de Componentes Principales)
 - **LDA** (Análisis Discriminante Lineal)
 - **SVD** (Descomposición en Valores Singulares)

Métodos no lineales

- Buscan descubrir estructuras más complejas llamadas **manifolds**.
- **Características:**
 - Capturan relaciones no lineales.
 - Conservan mejor la estructura local.
 - Requieren mayor costo computacional.
- **Algoritmos representativos:**
 - Kernel PCA
 - Isomap
 - LLE (Locally Linear Embedding)
 - t-SNE
 - UMAP
 - Autoencoders (redes neuronales)

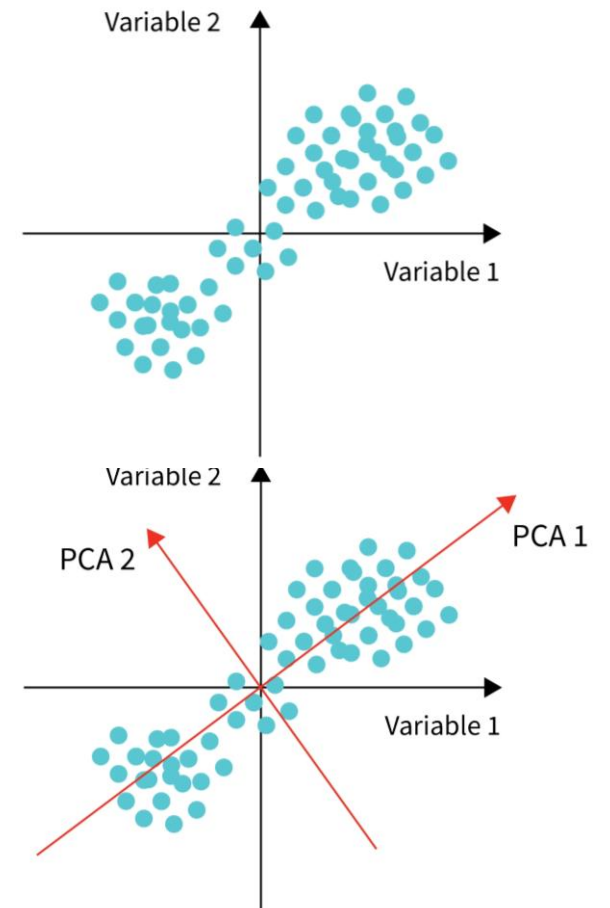
Métodos lineales vs no lineales

- **¿Cuándo usar cada uno?**
- **Métodos lineales**
 - Variables altamente correlacionadas.
 - Compresión.
 - Interpretabilidad.
 - Grandes volúmenes de datos.
- **Métodos no lineales**
 - Visualización.
 - Datos con estructuras complejas (alta dimensionalidad).
 - Embeddings.
 - Imágenes.
 - Texto.



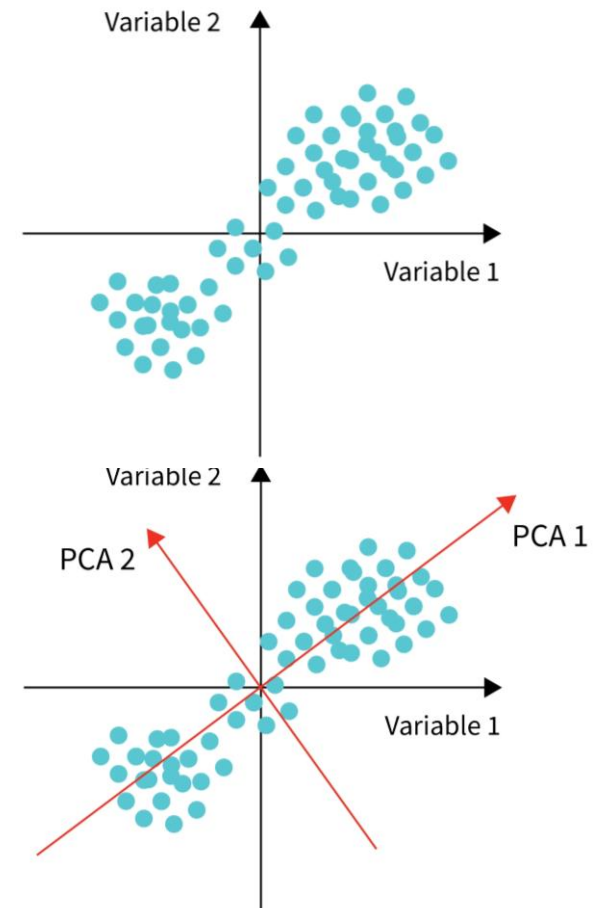
PCA (Principal Component Analysis)

- Es uno de los métodos más utilizados para reducir la dimensionalidad de un conjunto de datos.
- La idea principal consiste en encontrar las **direcciones donde los datos presentan la mayor variabilidad**.
- En lugar de conservar las variables originales, PCA construye un nuevo sistema de coordenadas formado por los **componentes principales**.



PCA (Principal Component Analysis)

- La mayor parte de la información se encuentra sobre una única dirección.
- En lugar de almacenar ambas variables, podemos proyectar los datos sobre esa dirección y conservar casi toda la información.
- **¿Qué busca PCA?**
 - Maximizar la varianza.
 - Reducir la redundancia.
 - Mantener la mayor cantidad posible de información.
 - Obtener nuevas variables independientes entre sí.



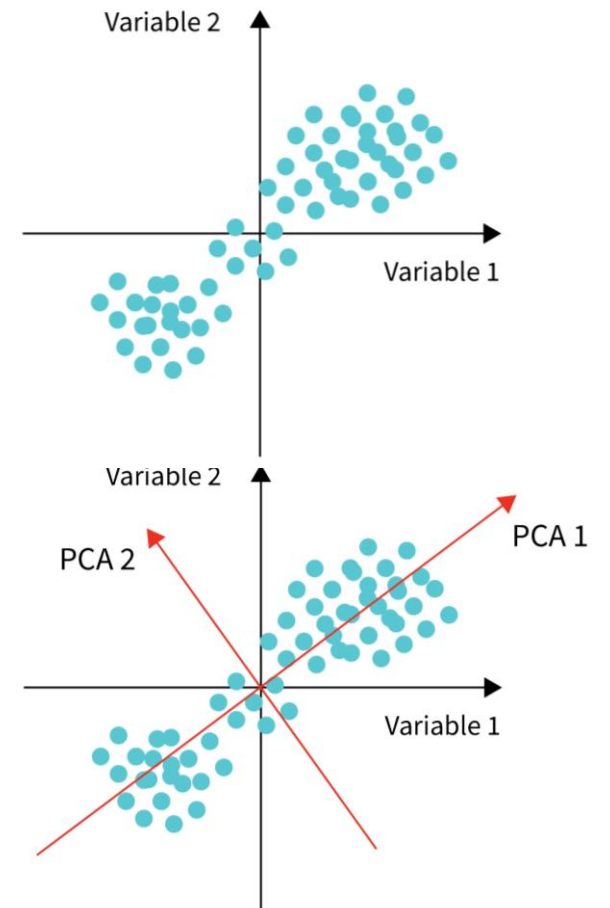
PCA (Principal Component Analysis)

- **Componentes principales**

- PCA obtiene nuevas variables llamadas **Componentes Principales**.
- Cada componente es una combinación lineal de las variables originales.

- **Matriz de covarianza**

- Antes de encontrar los componentes principales es necesario medir cómo varían conjuntamente las variables.
- La matriz de covarianza resume estas relaciones.
 - Covarianza positiva → ambas variables aumentan juntas.
 - Covarianza negativa → una aumenta mientras la otra disminuye.
 - Covarianza cercana a cero → poca relación lineal.



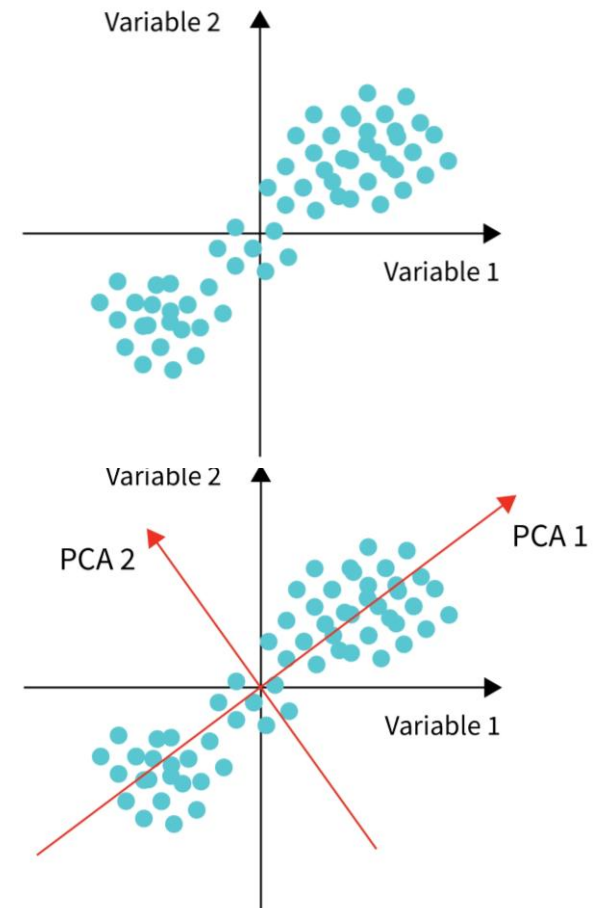
PCA (Principal Component Analysis)

- **Eigenvectores**

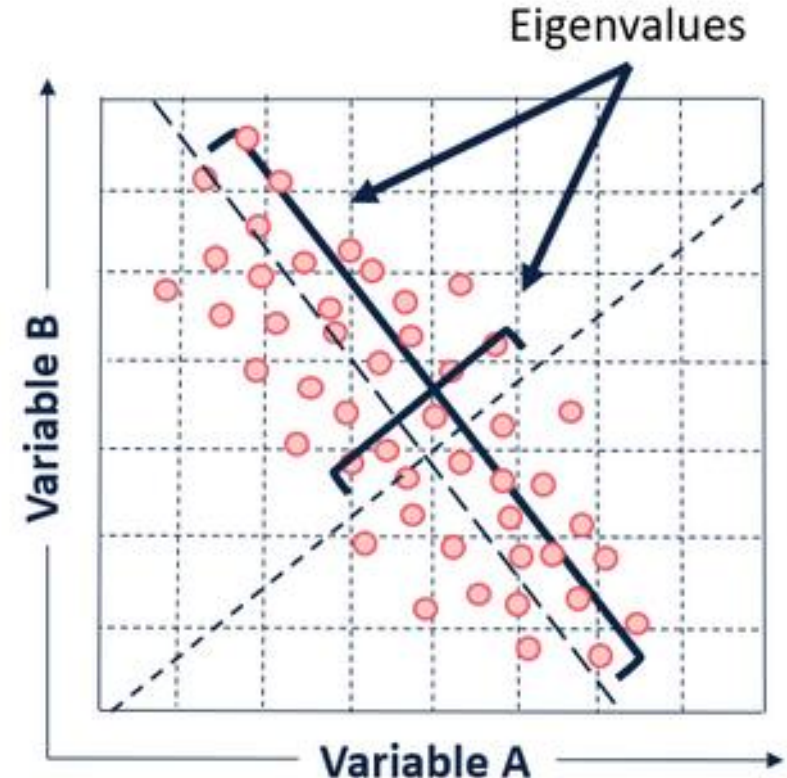
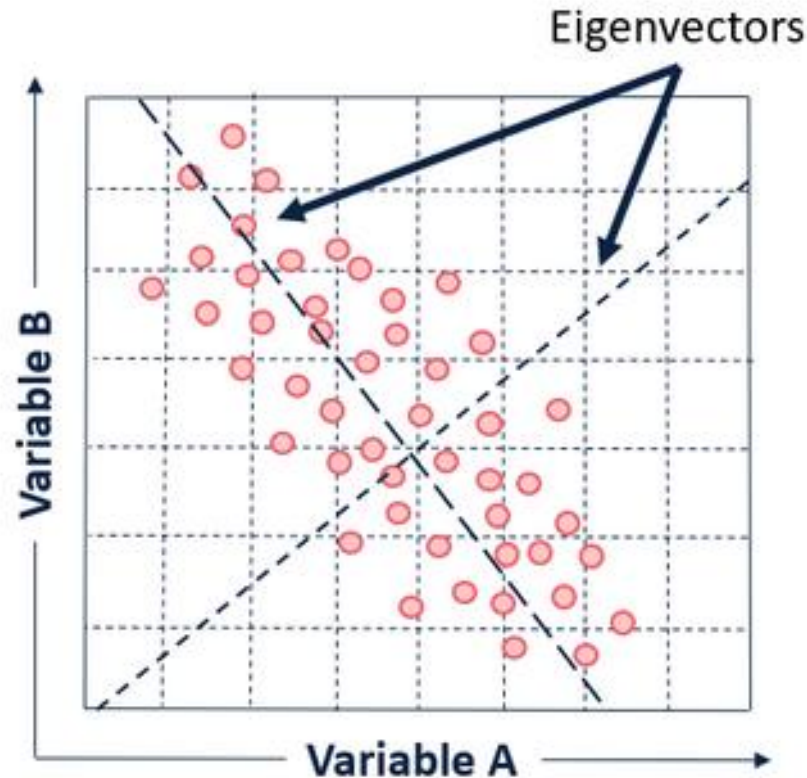
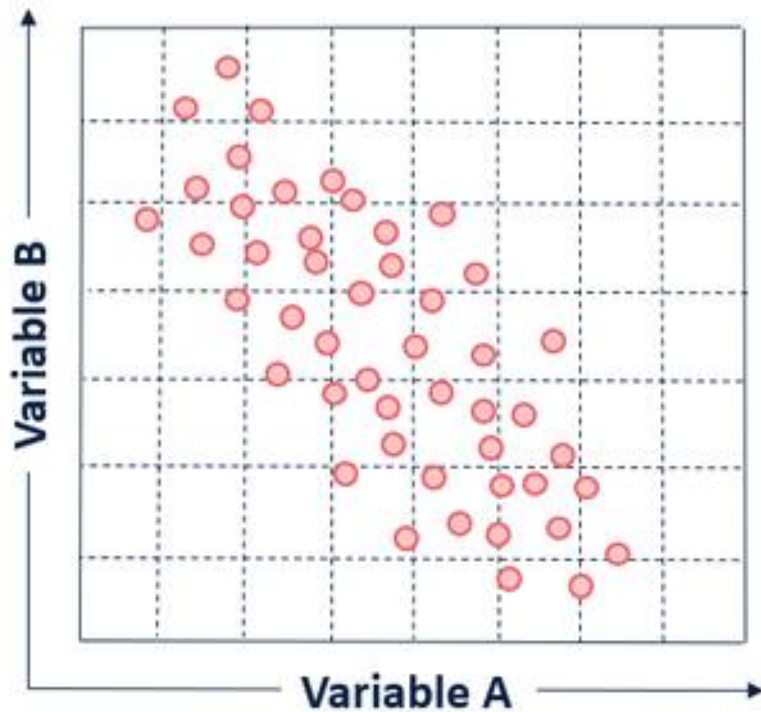
- Los **eigenvectores** indican las direcciones donde se proyectarán los datos.
- Cada eigenvector corresponde a una componente principal.
 - PC1 → máxima varianza.
 - PC2 → segunda máxima varianza.
- Todos son ortogonales entre sí.

- **Eigenvalores**

- Cada eigenvalor mide cuánta información (varianza) explica su componente principal.
 - Eigenvalor grande → componente importante.
 - Eigenvalor pequeño → poca información.
- Los componentes se ordenan de mayor a menor eigenvalor.



PCA (Principal Component Analysis)



Algoritmo de PCA

- Sea $X \in \mathbb{R}^{n \times p}$ una matriz con n observaciones y p variables.
- **1. Centrado de los datos**
- Se calcula la media de cada variable j :
$$\mu_j = \frac{1}{n} \sum_{i=1}^n x_{ij}$$
- Después, se resta esa media a cada valor de la variable:
$$x_{ij}^c = x_{ij} - \mu_j$$
- El centrado permite que PCA estudie la variación de los datos alrededor de su media.
- **Si las variables tienen unidades o escalas muy diferentes**, puede ser conveniente estandarizarlas, además de centrarlas.

Algoritmo de PCA

- **2. Cálculo de la matriz de covarianza**
- A partir de los datos centrados, se obtiene la matriz de covarianza muestral:
$$C = \frac{1}{n-1} X_c^T X_c$$
- La matriz $C \in \mathbb{R}^{p \times p}$ contiene las covarianzas entre todos los pares de variables.
- Su interpretación es:
 - **Diagonal:** C_{jj} es la varianza de la variable j .
 - **Covarianza positiva:** ambas variables tienden a aumentar o disminuir juntas.
 - **Covarianza negativa:** cuando una aumenta, la otra tiende a disminuir.
 - **Covarianza cercana a cero:** indica poca asociación lineal. No implica necesariamente independencia.

Algoritmo de PCA

- **3. Descomposición espectral**

- Se calculan los eigenvectores y eigenvalores de la matriz de covarianza:

$$C\mathbf{v}_j = \lambda_j\mathbf{v}_j$$

- donde:
 - $\mathbf{v}_j$ es un **eigenvector**: define la dirección de una componente principal.
 - λ_j es su **eigenvalor**: representa la varianza de los datos al proyectarlos sobre esa dirección.
- Los eigenvectores se eligen unitarios y mutuamente ortogonales:

$$\mathbf{v}_j^T \mathbf{v}_j = 1, \mathbf{v}_j^T \mathbf{v}_k = 0 \ (j \neq k)$$

Algoritmo de PCA

- Se ordenan los eigenvalores de mayor a menor, conservando la correspondencia con sus eigenvectores:

$$\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p \geq 0$$

- La primera componente captura la mayor varianza posible. La segunda captura la mayor varianza restante bajo la restricción de ser ortogonal a la primera, y así sucesivamente.
- La **proporción de varianza explicada** por la componente j es:

$$r_j = \frac{\lambda_j}{\sum_{\ell=1}^p \lambda_{\ell}}$$

- La varianza explicada acumulada por las primeras k componentes es:

$$R_k = \frac{\sum_{j=1}^k \lambda_j}{\sum_{\ell=1}^p \lambda_{\ell}}$$

Algoritmo de PCA

- Esto permite elegir cuántas componentes conservar. Por ejemplo, se puede seleccionar el menor k que cumpla:

$$R_k \geq 0.95$$

- Así se conserva al menos el 95% de la varianza total. Este umbral es una elección práctica, no una regla obligatoria.
- **4. Proyección sobre las componentes principales**
- Se construye una matriz con los eigenvectores correspondientes a los k mayores eigenvalores:

$$V_k = [\mathbf{v}_1 \quad \mathbf{v}_2 \quad \cdots \quad \mathbf{v}_k] \in \mathbb{R}^{p \times k}$$

Algoritmo de PCA

- Los **datos centrados** se proyectan sobre estas direcciones:

$$T = X_c V_k$$

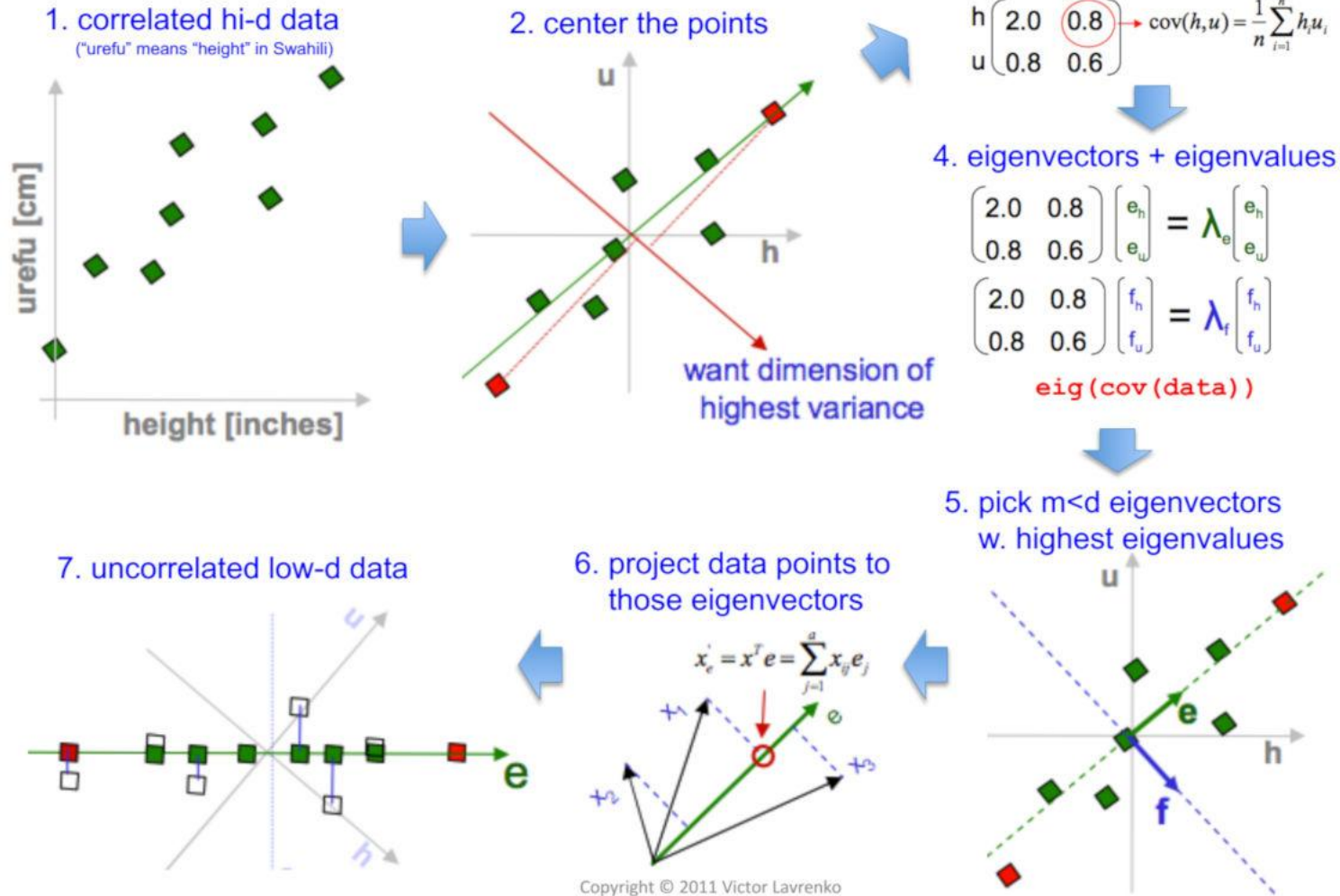
- La matriz $T \in \mathbb{R}^{n \times k}$ contiene las nuevas coordenadas de las observaciones. Cuando $k < p$, se reduce la dimensionalidad.
- El valor de la componente j para la observación i es:

$$t_{ij} = \sum_{\ell=1}^p (x_{i\ell} - \mu_{\ell}) v_{\ell j}$$

Algoritmo de PCA

- Por ejemplo, para tres variables, la primera componente se expresa como:
$$t_{i1} = v_{11}(x_{i1} - \mu_1) + v_{21}(x_{i2} - \mu_2) + v_{31}(x_{i3} - \mu_3)$$
- Las componentes resultantes tienen matriz de covarianza diagonal:
$$\frac{1}{n-1} T^T T = \text{diag}(\lambda_1, \dots, \lambda_k)$$
- Esto muestra que **las componentes no están correlacionadas entre sí** y que la varianza de cada una coincide con su eigenvalor.

PCA in a nutshell



Algoritmo de PCA

PCA (Principal Component Analysis)

- **Ventajas**

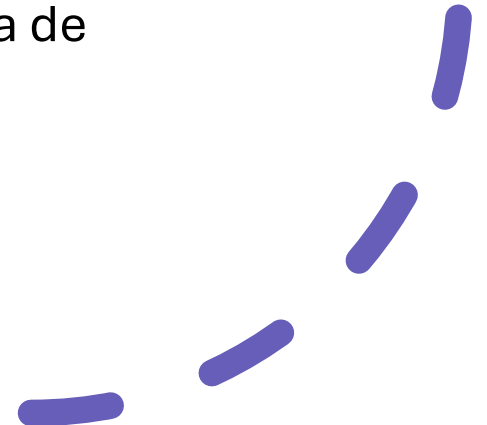
- Reduce significativamente la dimensionalidad.
- Elimina información redundante.
- Disminuye el costo computacional.
- Reduce el riesgo de sobreajuste.
- Facilita la visualización en dos o tres dimensiones.
- Produce componentes ortogonales (no correlacionadas).
- Fácil implementación y ampliamente disponible en librerías de Machine Learning (ej. scikit-learn).



PCA (Principal Component Analysis)

- **Limitaciones**

- Solo captura relaciones **lineales** entre las variables.
- **Puede perder información relevante** si se eliminan demasiadas componentes.
- Las nuevas variables son **difíciles de interpretar**.
- Es **sensible a la escala de las variables**, por lo que normalmente requiere normalización previa.
- Puede verse **afectado por valores atípicos** (outliers).
- No siempre la dirección de máxima varianza coincide con la dirección más útil para una tarea de clasificación.



Kernel PCA

- PCA funciona muy bien cuando los datos presentan una estructura aproximadamente **lineal**.
- Sin embargo, muchos conjuntos de datos reales contienen relaciones mucho más complejas.
- **¿Cuándo falla PCA?**
 - Cuando los datos:
 - Forman curvas.
 - Espirales.
 - Superficies curvas.
 - Estructura no lineal.
- En estos casos, una proyección lineal no logra separar correctamente los datos.



Kernel PCA

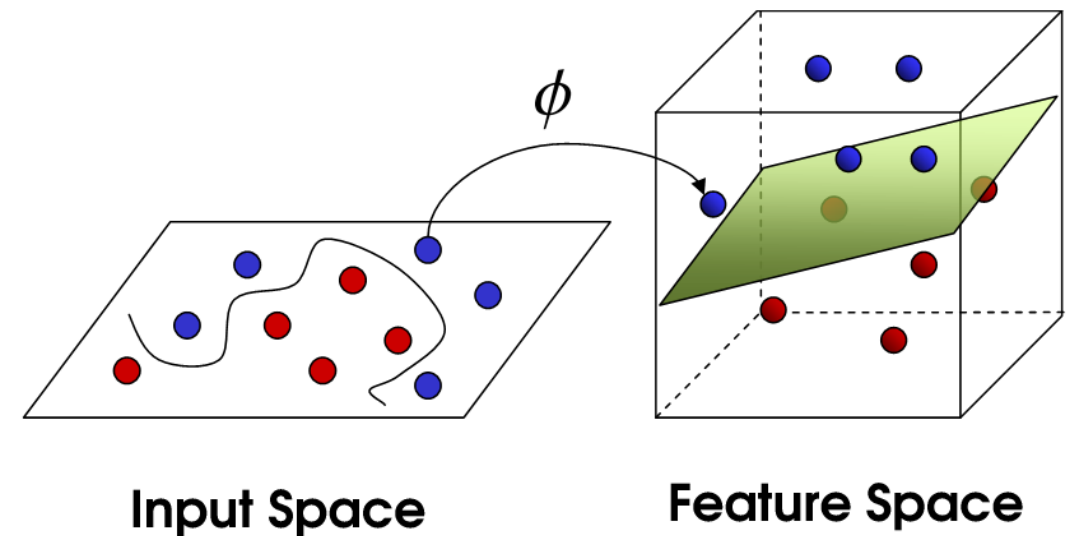
- En lugar de buscar componentes principales en el espacio original, podemos transformar los datos a un espacio donde la separación sea aproximadamente lineal.
- Esa es la idea detrás de **Kernel PCA**.
- Kernel PCA permite aplicar PCA a problemas donde las relaciones entre las variables son no lineales.



Kernel PCA

- **Kernel Trick**

- La idea central de Kernel PCA consiste en transformar los datos hacia un espacio de mayor dimensión donde las relaciones complejas se vuelvan aproximadamente lineales.
- Después, PCA se aplica en ese nuevo espacio.
- Transformar explícitamente cada punto puede ser extremadamente costoso.
- En lugar de calcular la transformación completa, únicamente se calcula la similitud entre pares de observaciones mediante una función Kernel.
- De esta forma se obtiene el mismo resultado sin construir explícitamente el nuevo espacio.



Kernel PCA

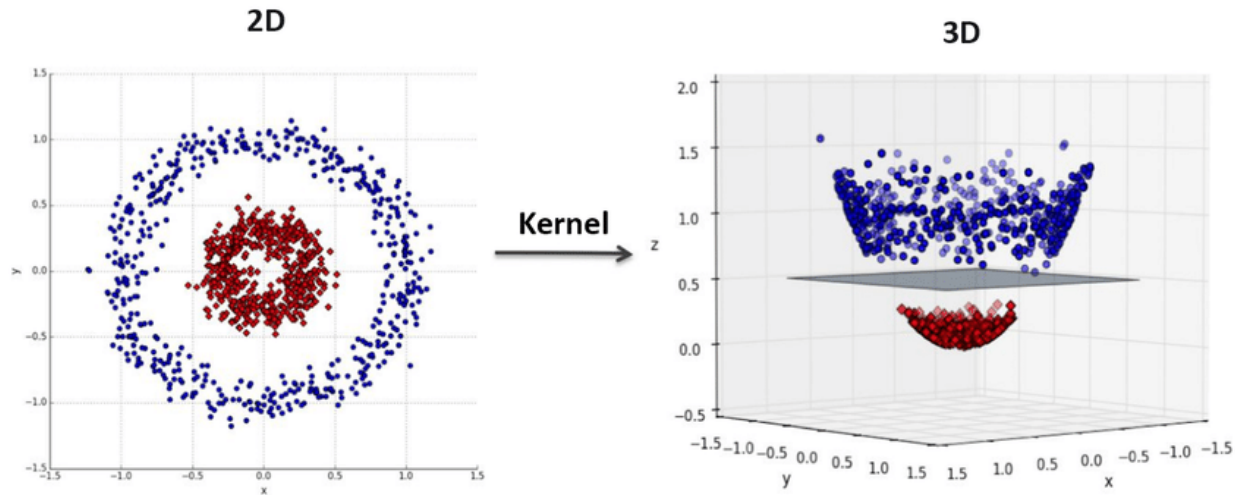
- **Kernels más utilizados:**

- Kernel RBF (Radial Basis Function)

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$$

- Kernel Polinomial

$$K(x_i, x_j) = (x_i^T x_j + c)^d$$



Kernel PCA

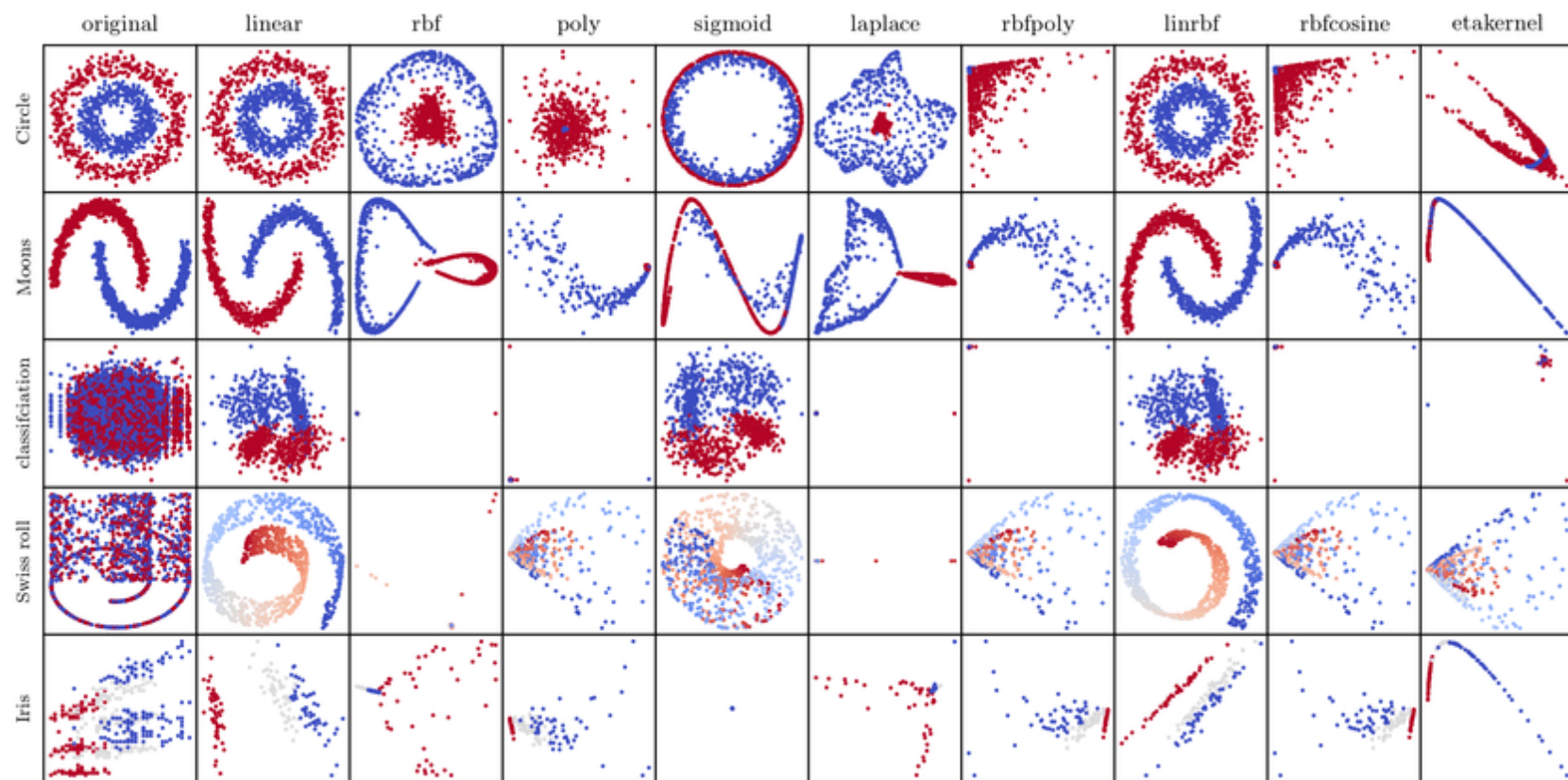
- **Ventajas**

- Captura relaciones no lineales.
- Permite representar datos complejos con pocas dimensiones.
- Excelente para visualización y preprocesamiento.

- **Limitaciones**

- Mayor costo computacional que PCA.
- La elección del Kernel y sus parámetros es muy importante.
- Menor interpretabilidad de las componentes principales.
- No siempre existe un Kernel adecuado para todos los problemas.

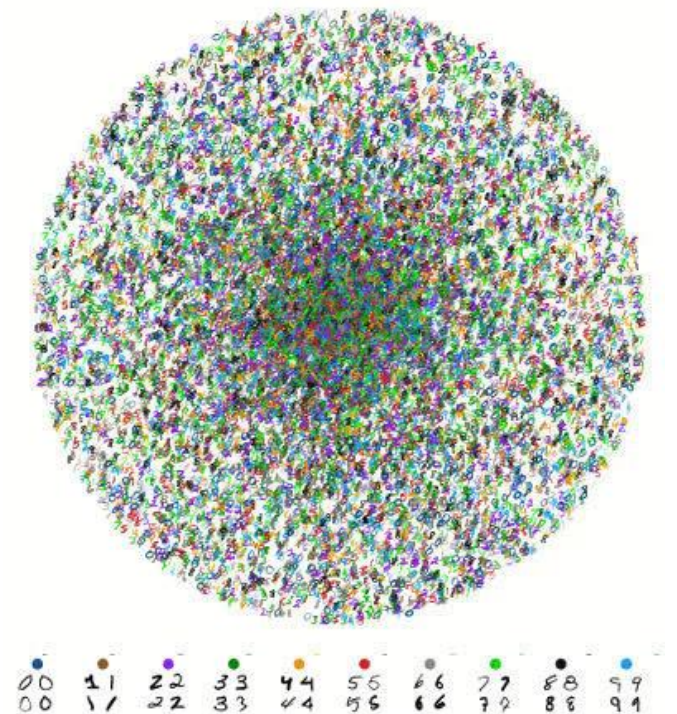




Kernel PCA

t-SNE (t-Distributed Stochastic Neighbor Embedding)

- t-SNE es un método **no lineal** diseñado principalmente para **visualizar datos de alta dimensión** en dos o tres dimensiones.
- Conserva las relaciones entre vecinos cercanos.
- Convierte las distancias entre observaciones en probabilidades y busca mantener esas probabilidades en el espacio reducido.
- **Ventajas**
 - Excelente para visualizar clusters.
 - Descubre estructuras complejas.
- **Limitaciones**
 - No preserva bien las distancias globales.
 - Es costoso computacionalmente.
 - No es adecuado para proyectar nuevos datos.



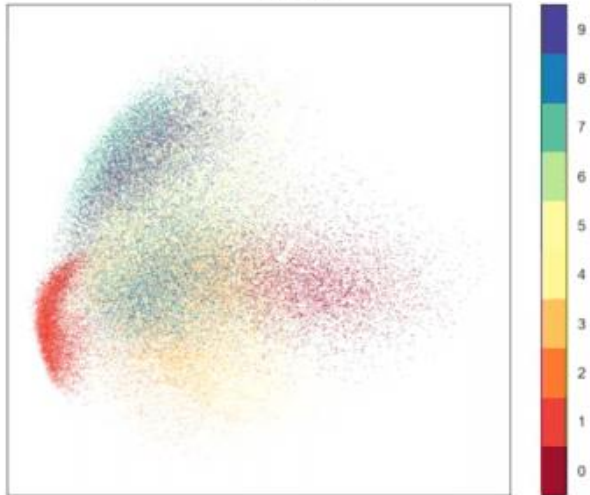


UMAP (Uniform Manifold Approximation and Projection)

- UMAP es un método **no lineal** basado en la hipótesis de que los datos se encuentran sobre un **manifold** de menor dimensión.
 - Construye un grafo de vecinos y busca conservar tanto la estructura local como parte de la estructura global.
 - **Ventajas**
 - Muy rápido.
 - Escalable a grandes conjuntos de datos.
 - Conserva mejor la estructura global que t-SNE.
 - Permite proyectar nuevos datos.
 - **Limitaciones**
 - Depende de la elección de sus parámetros.
 - La interpretación de los ejes sigue sin tener significado físico.
-

MNIST Digits

PCA



t-SNE

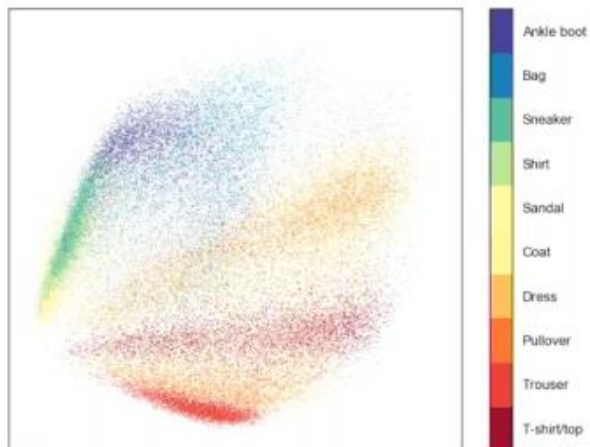


UMAP

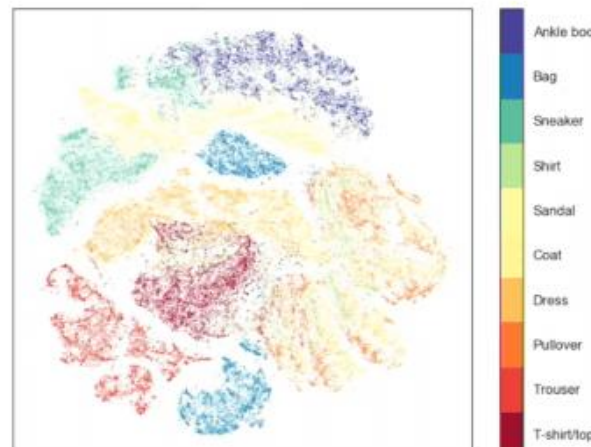


Fashion MNIST

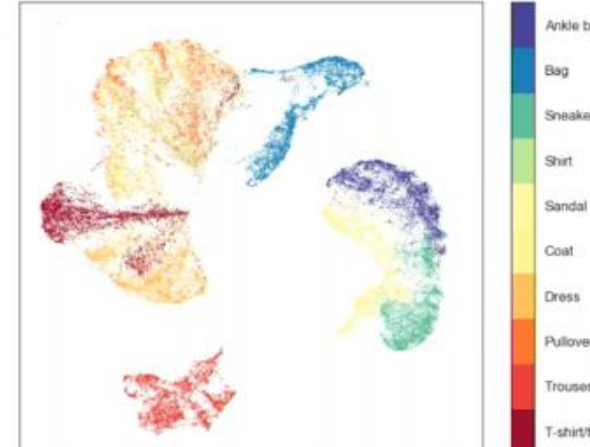
PCA



t-SNE

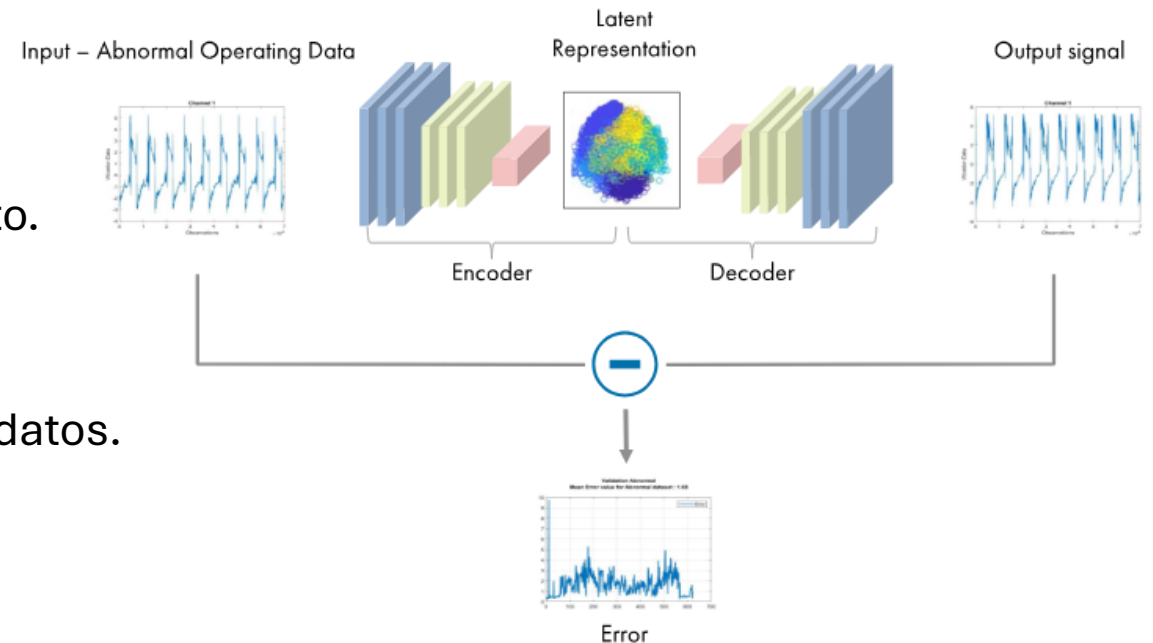


UMAP



Autoencoders

- Los **Autoencoders** son redes neuronales diseñadas para aprender una representación comprimida de los datos.
- **Ventajas**
 - Capturan relaciones altamente no lineales.
 - Funcionan muy bien con imágenes, audio y texto.
 - Escalan a grandes volúmenes de datos.
- **Limitaciones**
 - Requieren entrenamiento y mayor cantidad de datos.
 - Mayor costo computacional.
 - Menor interpretabilidad que métodos clásicos.



Conclusiones

- La **reducción de dimensionalidad** permite representar datos complejos utilizando menos variables, preservando la mayor cantidad posible de información.
- **PCA** es el método lineal más utilizado y constituye un excelente punto de partida cuando existen variables correlacionadas.
- Cuando los datos presentan **relaciones no lineales**, métodos como **Kernel PCA**, **t-SNE**, **UMAP** y **Autoencoders** pueden capturar estructuras que PCA no puede representar.
- La reducción de dimensionalidad es una herramienta fundamental para **visualización**, **compresión**, **eliminación de ruido**, **extracción de características**.

Muchas gracias

jraygoza@cicese.edu.mx
